

2026 开源鸿蒙大学生创新大赛-赛道一

赛题：OpenHarmony Agent 鸿蒙原生智能多模态个人助理

学校名称： 按报名系统填写

团队名称： 浮伴团队

队长： 按报名系统填写

队员 1： 按报名系统填写

2026 开源鸿蒙大学生创新大赛组委会

目 录

1简介	3
1.1 背景	3
1.2 目的	3
2设计描述	3
2.1 总体设计	3
2.2 实现思路	3
2.3 系统结构	3
2.3.1 模块划分	3
2.3.2 系统架构说明	3
2.3.3 文件结构	3
2.4 模块功能描述	3
2.4.1 模块/子系统 1 功能描述	4
2.4.2 模块/子系统 2 功能描述	4
2.4.3 模块/子系统 3 功能描述	4
2.5 业务/实现流程说明	4
2.5.1 用例 1 处理流程	4
2.5.2 用例 2 处理流程	4
2.6 接口描述	4
2.6.1 调用接口	5
2.6.2 提供接口	5
2.7 UI 设计	5
3其他	5
3.1 成员分工	5
3.2 困难与思考	5
3.3 参考	5

1 简介

1.1 背景

桌面工作中的报错、信息确认和上下文切换往往打断用户。浮伴将 MiniCPM-o 的视觉、语音和文本能力放到桌面陪伴形态中，平时安静存在，用户明确授权后才开启对应输入，并在当前会话内提供协助。本开发设计文档用于说明浮伴 Windows Electron 客户端、模型适配服务以及 OpenHarmony NEXT PC 延伸工程的设计与实现边界。

1.2 目的

本文档给出浮伴的总体设计、模块划分、关键流程、接口边界和 UI 组织方式，为代码实现、测试验收和后续 OpenHarmony 适配提供统一依据。文档同时区分真实模型链路与离线 Fake 演示，避免把演示占位行为误认为模型效果。

2 设计描述

浮伴采用分层拆分：Electron 主进程负责窗口、托盘和生命周期；Renderer 负责桌宠、Dock、设置和业务交互；Preload 只暴露白名单安全桥；Python 服务负责把 Chat 与 Realtime 请求适配到 MiniCPM-o。这样可以在模型服务不可用时保留完整客户端演示，也便于将 Renderer 复用到 OpenHarmony ArkWeb。

2.1 总体设计

系统提供透明置顶桌宠、受控多模态输入、主动询问、协助卡、多轮会话、记忆管理、任务、便签、专注计时、托盘控制和隐私收口。默认状态为感知关闭，麦克风、摄像头和指定屏幕分别授权；用户可暂停采集、开启勿扰或结束陪伴。

2.2 实现思路

以用户控制权为主线：先由桌宠保持低打扰常驻，再由用户明确开启会话和输入来源；重复线索经过记录与再次确认后才发起询问；用户接受后将当前上下文送入 Chat 或 Realtime 链路，并把回复回传到协助卡。离线时使用带 Fake 标识的确定性流程验证 UI 和状态机，真实能力需在模型服务环境单独验收。

2.3 系统结构

2.3.1 模块划分

模块划分如下：1) 桌面容器层：Electron 主进程、窗口管理、托盘、单实例和生命周期；2) 交互呈现层：Renderer 中的桌宠、Dock、状态条、设置、协助卡、任务、便签和专注计时；3) 安全桥接层：Preload 白名单 API、媒体权限和 IPC；4) 模型适配层：Python 服务、Chat 请求、Realtime 会话、音频与 JPEG 编码；5) 跨端适配层：HarmonyOS ArkWeb Renderer 与 ArkTS 系统能力桥。

2.3.2 系统架构说明

启动时主进程创建透明窗口并加载 Renderer, Preload 建立受限 IPC。用户点击开始后进入陪伴状态但不自动采集；用户逐项授权输入后，Renderer 将媒体状态交给主进程和适配服务。Chat 采用请求-响应返回文字、JPEG 与短 WAV 结果；Realtime 采用 16 kHz 音频输入、JPEG 视觉帧、增量文字和 24 kHz 音频输出。退出、暂停采集或结束陪伴时，主进程停止媒体轨道并关闭实时连接。

2.3.3 文件结构

主要目录：desktop/ 为 Electron 客户端、preload 和测试；desktop/src/renderer/ 为桌宠与业务 UI；desktop/src/main/ 为窗口、托盘和 IPC；service/ 为 Python 模型适配服务；harmony/ 为 OpenHarmony NEXT PC 工程和 Renderer 同步脚本；web-demo/ 为静态离线 Web 演示；submission/ 为宣传页、视频和提交文档；docs/ 为产品设计、部署与验收记录。

2.4 模块功能描述

本节按职责描述桌面容器、交互呈现和模型适配三个核心模块，并说明它们的输入、输出和边界。模块功能按桌面容器、交互呈现和模型适配三类核心职责展开，下面给出各模块的目的、功能和处理边界。

2.4.1 模块/子系统 1 功能描述

标识：桌面容器层。类型：Electron 主进程模块。目的：提供稳定的桌面宿主和系统级生命周期控制。功能列表：创建透明置顶窗口、跳过任务栏、单实例、托盘菜单、窗口位置和退出清理。处理：主进程集中处理窗口与系统能力，Renderer 通过 Preload 白名单 API 请求，不直接访问 Node.js 或文件系统。

标识：交互呈现层。类型：Renderer 业务 UI 与本地状态模块。目的：呈现桌宠状态并承载用户可见的协助流程。功能列表：桌宠与 Dock、设置、状态条、协助卡、记忆、任务、便签和专注计时。处理：由统一状态驱动 UI，使用本地存储保存用户确认过的记忆和计时结束时间，重启后恢复可见状态。

标识：模型适配层。类型：Python 服务与协议适配模块。目的：把客户端输入转换为 MiniCPM-o 可处理的 Chat/Realtime 请求。功能列表：文字、JPEG、WAV、16 kHz 音频输入、增量文字和 24 kHz 音频输出。处理：服务异常时返回明确错误，客户端保持 Fake/离线标识，不把固定回复写成真实推理。

2.4.2 模块/子系统 2 功能描述

交互呈现层负责桌宠、Dock、设置、协助卡和本地业务状态。状态变化统一由 Renderer store 驱动，记忆、任务、便签和专注计时使用本地存储；重启后专注计时根据结束时间重新计算剩余时间。

2.4.3 模块/子系统 3 功能描述

模型适配层负责 Chat/Realtime 协议转换、媒体编码和错误回收。Chat 支持文字、JPEG 和短 WAV；Realtime 支持 16 kHz 音频、JPEG 帧、增量文字和 24 kHz 音频输出。

2.5 业务/实现流程说明

主动协助和实时对话都采用“授权—处理—可停止—释放资源”的顺序，以下分别说明代表性用例。业务流程按两个代表性用例展开：主动协助和实时对话。两者都遵循开始会话、明确授权、建立输入、处理结果、用户可停止、释放资源的顺序。

2.5.1 用例 1 处理流程

用户开始陪伴后仍保持输入关闭；授权来源后，Renderer 记录线索并在重复确认后询问。用户接受才打开协助卡并调用 Chat；拒绝则不建立模型请求。

2.5.2 用例 2 处理流程

用户显式开启实时对话后建立 Realtime 连接，持续发送音频和可选视觉帧；增量结果更新 UI。点击停止、暂停采集或结束陪伴时，媒体轨道和连接立即释放。

2.6 接口描述

2.6.1 调用接口

主要调用接口：Electron BrowserWindow、Tray、screen、desktopCapturer；浏览器 MediaDevices 的麦克风/摄像头/屏幕采集接口；Python 适配服务的 Chat 与 Realtime HTTP/WebSocket 接口；OpenHarmony ArkTS/ArkWeb 能力桥接接口。调用时均以显式授权、状态校验和错误回收为前提。

2.6.2 提供接口

主要提供接口：Preload 向 Renderer 提供会话控制、输入开关、Dock 操作、记忆、任务、便签和专注计时 API；模型适配服务提供 Chat 请求、Realtime 建连、输入帧发送、增量文字和音频输出事件；HarmonyOS 工程提供 Renderer 同步和系统能力桥接入口。

2.7 UI 设计

UI 采用透明桌宠主视图加按需展开的 Liquid Glass Dock、状态条、设置面板和协助卡。常用任务、便签和设置从底栏直接进入，避免遮挡宠物主体；对话区保持足够宽度，气泡使用宠物指向三角但不覆盖正文。所有关键状态同时提供文字与颜色/角标反馈，并保留键盘焦点、可见焦点和高对比度可读性。

3 其他

3.1 成员分工

成员分工：产品与交互、Electron 客户端、Python 模型服务、OpenHarmony 适配和测试工作按报名团队实际成员填写；本模板不代填个人姓名。

3.2 困难与思考

困难与思考：主要难点是透明桌宠与普通窗口的交互边界、媒体权限的即时收口、Realtime 资源释放以及 Fake 与真实模型能力的事实区分。设计上优先采用白名单桥接、默认关闭输入、显式停止和可观察状态，降低误采集与误解风险。

3.3 参考

参考：MiniCPM-o 4.5 项目资料与已验证的 Chat/Realtime 服务说明；OpenHarmony NEXT PC 工程文档；Electron 官方 API 文档；仓库中的 docs/ 产品设计、部署和验收记录。